Ansible Automation - Use Cases

A reference of the use cases by which Ansible Automation Platform is commonly implemented, by domain. Each use case is described in the same format (Description, Collection, Key content, Notes) and mapped to Red Hat Certified and Validated content.

4 solution use cases · 0 product use cases · generated 2026-10-21 · community reference, not an official Red Hat publication

Contents

1. Reporting & Analytics
    - Solution use cases (4)
        - Report on job activity and host coverage
        - Measure automation value and ROI
        - Generate configuration and inventory reports
        - Report compliance status

Reporting & Analytics

Measure automation value and report on the state of managed estates.

Solution use cases

Use Case: Report on job activity and host coverage

Description	Answer the questions management asks about automation: how many jobs ran, how many succeeded, who used the platform, how long jobs took, and how much of the estate is actually touched by automation. Ansible reads the platform's own data, builds the report on a schedule and sends it to the people who need it.
Collection	`ansible.controller` (Certified), `ansible.platform` (Certified), `infra.aap_configuration` (Validated)
Key content	• `ansible.controller.job_list` — List Automation Platform Controller jobs. • `ansible.platform.job_list` — List jobs from Automation Controller. • `ansible.controller.controller_api` — Search the API for objects • `infra.aap_configuration.get_stats` — Get stats • `ansible.controller.schedule` — create, update, or destroy Automation Platform Controller schedules. • `ansible.controller.notification_template` — create, update, or destroy Automation Platform Controller notification.
Notes	• The platform's built-in dashboards cover day-to-day views. Build a custom report when the audience needs different grouping, such as by business unit, or data merged with other sources. • Keep job history retention (cleanup jobs) aligned with the reporting period, otherwise old figures disappear. • Use Measure automation value and ROI to turn these counts into time and cost savings. • Pair with Monitor Automation Platform health and capacity for operational rather than business reporting.

Use Case: Measure automation value and ROI

Description	Show the business what automation returns. For each automated process the team records how long and how costly the manual version was, and Ansible multiplies that by the number of successful runs. The result is a regular statement of hours saved, cost avoided and, where change or incident data is available, faster lead times.
Collection	`ansible.controller` (Certified), `servicenow.itsm` (Certified)
Key content	• `ansible.controller.job_list` — List Automation Platform Controller jobs. • `ansible.controller.controller_api` — Search the API for objects • `ansible.controller.label` — create, update, or destroy Automation Platform Controller labels. • `servicenow.itsm.change_request_info` — List ServiceNow change requests • `servicenow.itsm.incident_info` — List ServiceNow incidents

Notes	• Use conservative baselines and count only successful runs. A defensible small number is more useful than a large one that is challenged. • Not all value is time: consistency, audit evidence and speed are better measured with change and incident data. • Start from the activity report in Report on job activity and host coverage and consider hosts automated per process, not only job counts. • Unused templates in the report are a prompt for clean-up; see Assess the current automation estate.

Use Case: Generate configuration and inventory reports

Description	Replace spreadsheets and one-off scripts with a repeatable report of what is deployed: devices, versions, interfaces, addresses, installed software and configuration items. Ansible gathers the facts, stores them as structured data, renders HTML and CSV for readers, and keeps the configuration management database in step with what is really there.
Collection	`network.inventory_reports` (Validated), `network.base` (Validated), `servicenow.itsm` (Certified), `cisco.ios` (Certified), `arista.eos` (Certified), `cisco.iosxr` (Certified), `cisco.nxos` (Certified)
Key content	• `network.inventory_reports.gather` — The `gather` role collects comprehensive network device facts, including hardware details using native parsers. • `network.inventory_reports.persist` — The `persist` role enables users to fetch facts for specified network resources and store them in a YAML-formatted structure. These host variables can be saved either locally or in a remote data store, acting as a single source of truth (SOT) for network configurations. • `network.inventory_reports.generate_report` — The `generate_report` role allows users to generate HTML reports based on collected network device facts. This role parses the gathered facts and presents them in a visually structured HTML format. • `network.inventory_reports.flatten_dict` — Prepares structured data for CSV export. • `servicenow.itsm.configuration_item_batch` — Manage ServiceNow configuration items in batch mode • `servicenow.itsm.configuration_item_info` — List ServiceNow configuration item
Notes	• The network roles currently cover a defined set of platforms, so check the supported platform list before promising coverage of a vendor. • Related: Back up device configuration stores configuration text, whereas this use case stores structured facts. • Pair with Report compliance status: the same stored facts are the baseline for drift detection. • Event-Driven Ansible can refresh the inventory and CMDB after a change event, so reports do not wait for the next schedule.

Use Case: Report compliance status

Description	Give auditors and security teams a current, repeatable view of how systems compare with the policies the organization has chosen (for example CIS or DISA STIG profiles, or an approved device configuration). Ansible runs the scans on a schedule, collects the results from each tool, and produces one report with the failures and the owners to act on them.

Collection	`redhat.insights` (Certified), `infra.automation_development_office` (Validated), `network.inventory_reports` (Validated), `servicenow.itsm` (Certified)
Key content	• `redhat.insights.compliance` — Install and configure Red Hat Insights Client • `infra.automation_development_office.rhel_stig_cac` — Apply DISA STIG Compliance-as-Code on RHEL 8, 9, and 10 hosts. • `infra.automation_development_office.ocp_compliance_scan` — An collection. • `infra.automation_development_office.acs_report` — Generate RHACS vulnerability workload reports (raw/rhsource/age/all) • `network.inventory_reports.detect` — The `detect` role enables users to identify configuration drifts between the current running configurations and the desired state provided in a local or remote inventory. This role ensures that network devices remain compliant with their intended configurations. • `servicenow.itsm.incident` — Manage ServiceNow incidents
Notes	• Scanning and remediation are separate steps: report first, then remediate in a controlled change. See the security and Linux use cases for the remediation side. • Some roles in `infra.automation_development_office` are tailored to its bootstrap framework; check their variables before reuse. • Insights compliance needs the host registered and a policy set in the Insights service first; the role runs the scan on the host. • Use Event-Driven Ansible to start a rescan after a remediation or change, and to open an incident only when a control stays failed. • Combine with Report on job activity and host coverage so systems that were never scanned are visible too.